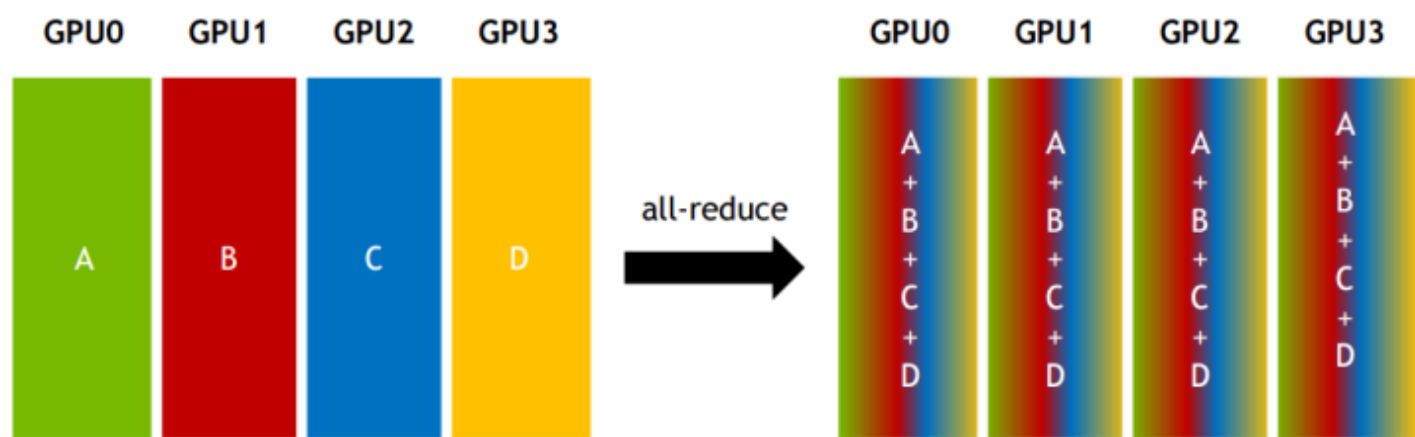


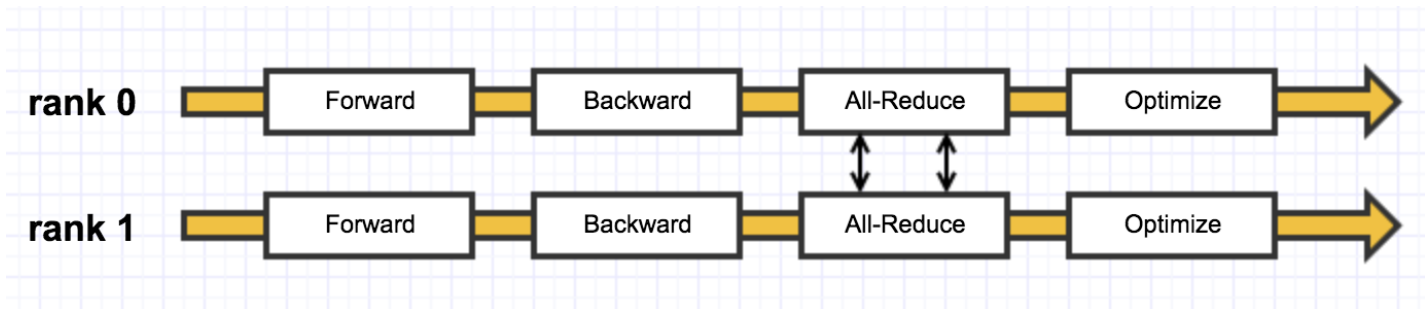
all-reduce算法

📄 本页面由边俊柯于2022/06/17迁移自wiki 陈庆澍空间

1. all reduce简介



all-reduce图解



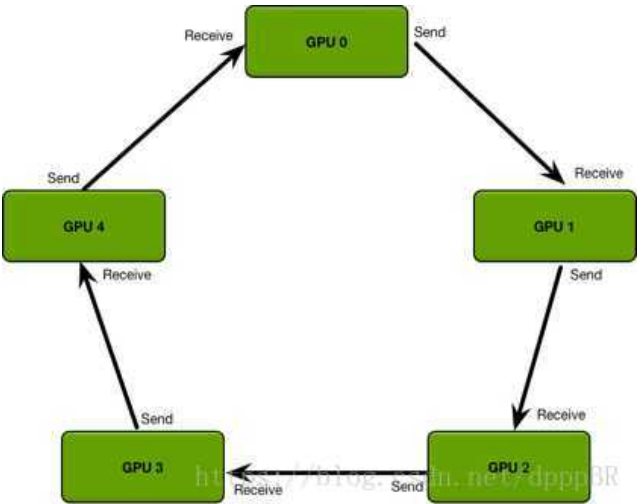
分布式训练调用all-reduce操作图解

2. 通讯Cost计算

假设 a 为发送一个消息需要的startup损失， b 为发送一个byte的传输耗时， n 为总共要发送的数据量，则每次通讯的cost为 $a + nb$

3. ring all reduce

ring all-reduce把所有的节点串成一个环，算法的进行分两步：第一步，scatter-reduce；第二步，allgather。在第一步中，GPU 将交换数据，使得每个 GPU 最终都有一个最终结果的数据块。在第二步中，GPU 将交换那些块，使得所有 GPU 最终得到完整的最后结果。



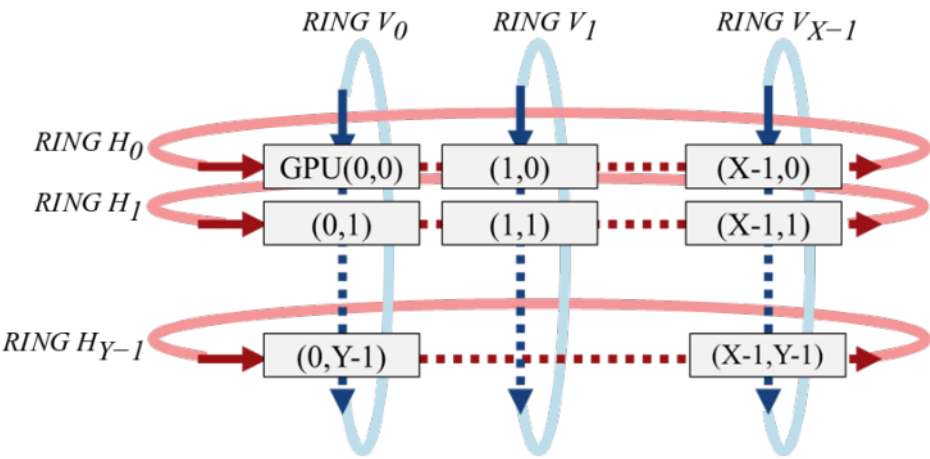
gpu0	a_0	b_0	c_0
gpu1	a_1	b_1	c_1
gpu2	a_2	b_2	c_2
gpu0	a_0	b_0	c_0+c_2
gpu1	a_0+a_1	b_1	c_1
gpu2	a_2	b_1+b_2	c_2
gpu0	a_0	$b_0+b_1+b_2$	c_0+c_2
gpu1	a_0+a_1	b_1	$c_0+c_1+c_2$
gpu2	$a_0+a_1+a_2$	b_1+b_2	c_2

ring all-reduce 是这样一种算法——其通信成本是恒定的，与系统中的 GPU 的数量无关，并且仅由系统中的 GPU 之间的最慢连接来确定。事实上，如果在通信成本上你只考虑带宽这一因素（并忽略延迟），那么 ring allreduce 就是一个最佳的通信算法。

利用章节1介绍的计算方法，假设有p个节点，则cost为 $2(p-1)a + 2(1-1/p)nb$

4. 2d-torus all-reduce

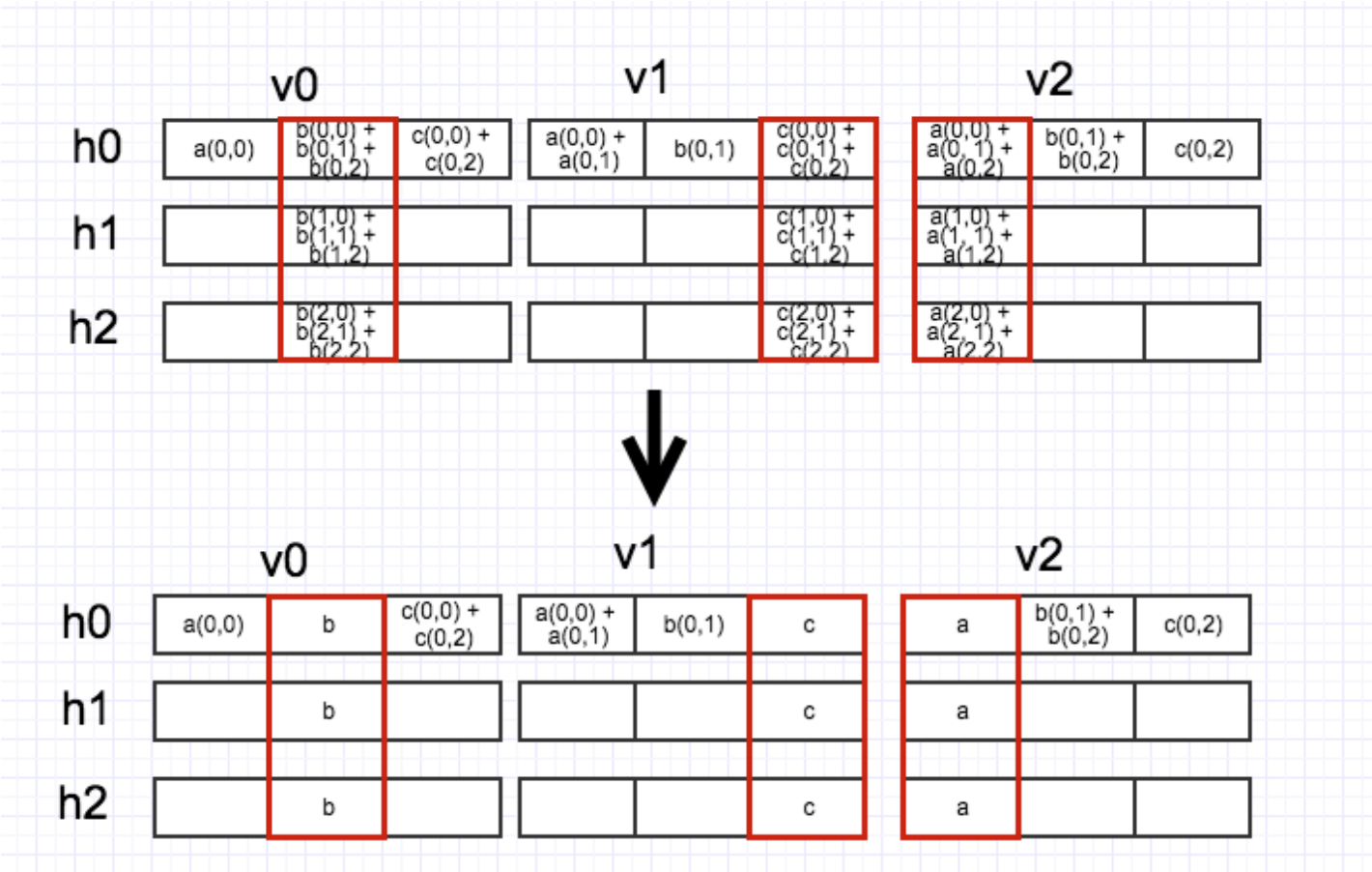
2d-torus all-reduce把节点分成横向的环和纵向的环，该算法的进行分成三步： 第一步，横向scatter-reduce； 第二步纵向all-reduce; 第三步： 横向all-gather



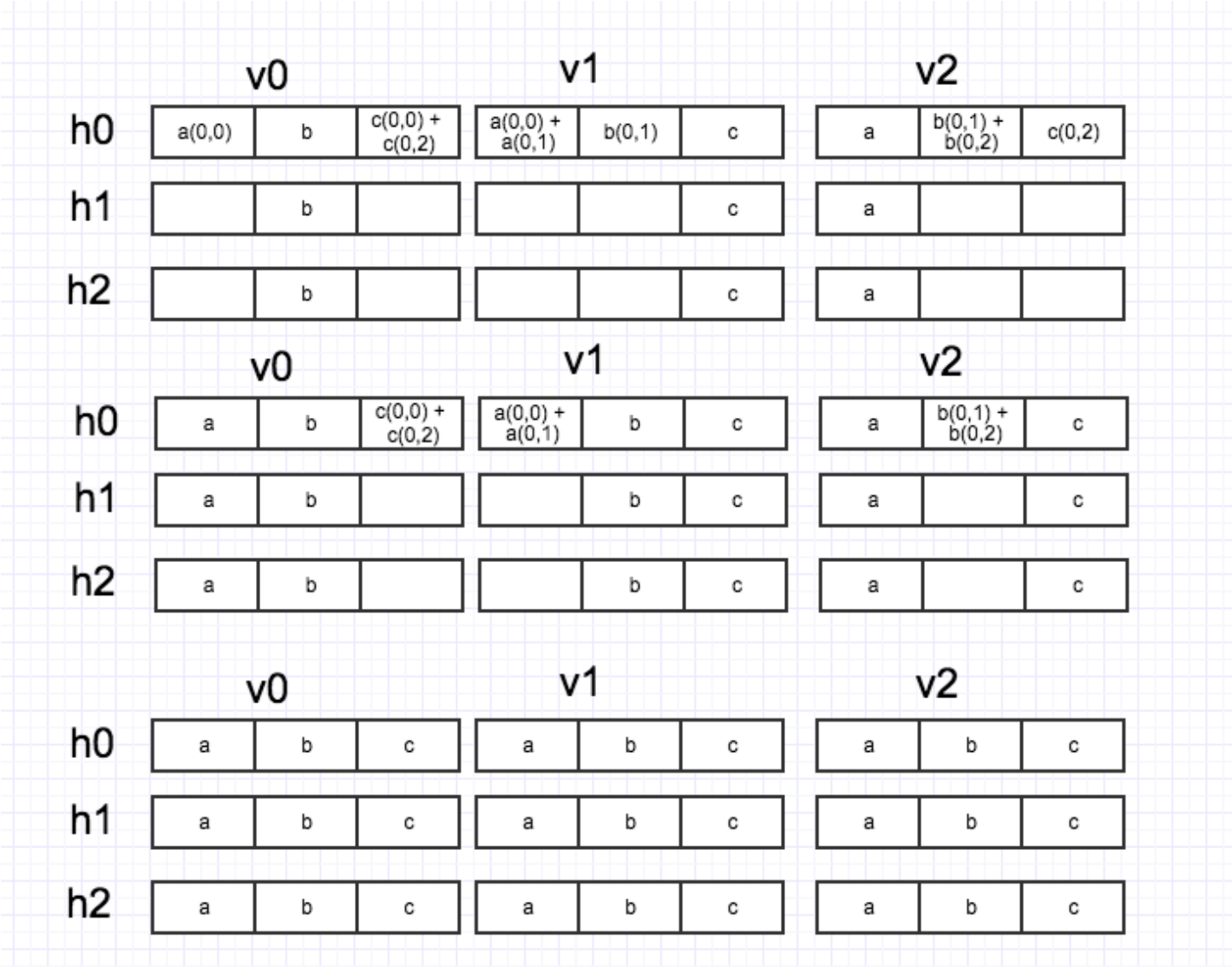
横向scatter-reduce:

	v0	v1	v2									
h0	<table><tr><td>a(0,0)</td><td>b(0,0)</td><td>c(0,0)</td></tr></table>	a(0,0)	b(0,0)	c(0,0)	<table><tr><td>a(0,1)</td><td>b(0,1)</td><td>c(0,1)</td></tr></table>	a(0,1)	b(0,1)	c(0,1)	<table><tr><td>a(0,2)</td><td>b(0,2)</td><td>c(0,2)</td></tr></table>	a(0,2)	b(0,2)	c(0,2)
a(0,0)	b(0,0)	c(0,0)										
a(0,1)	b(0,1)	c(0,1)										
a(0,2)	b(0,2)	c(0,2)										
h1	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
h2	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
	v0	v1	v2									
h0	<table><tr><td>a(0,0)</td><td>b(0,0)</td><td>c(0,0) + c(0,2)</td></tr></table>	a(0,0)	b(0,0)	c(0,0) + c(0,2)	<table><tr><td>a(0,0) + a(0,1)</td><td>b(0,1)</td><td>c(0,1)</td></tr></table>	a(0,0) + a(0,1)	b(0,1)	c(0,1)	<table><tr><td>a(0,2)</td><td>b(0,1) + b(0,2)</td><td>c(0,2)</td></tr></table>	a(0,2)	b(0,1) + b(0,2)	c(0,2)
a(0,0)	b(0,0)	c(0,0) + c(0,2)										
a(0,0) + a(0,1)	b(0,1)	c(0,1)										
a(0,2)	b(0,1) + b(0,2)	c(0,2)										
h1	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
h2	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
	v0	v1	v2									
h0	<table><tr><td>a(0,0)</td><td>b(0,0) + b(0,1) + b(0,2)</td><td>c(0,0) + c(0,1) + c(0,2)</td></tr></table>	a(0,0)	b(0,0) + b(0,1) + b(0,2)	c(0,0) + c(0,1) + c(0,2)	<table><tr><td>a(0,0) + a(0,1)</td><td>b(0,1)</td><td>c(0,0) + c(0,1) + c(0,2)</td></tr></table>	a(0,0) + a(0,1)	b(0,1)	c(0,0) + c(0,1) + c(0,2)	<table><tr><td>a(0,0) + a(0,1) + a(0,2)</td><td>b(0,1) + b(0,2)</td><td>c(0,2)</td></tr></table>	a(0,0) + a(0,1) + a(0,2)	b(0,1) + b(0,2)	c(0,2)
a(0,0)	b(0,0) + b(0,1) + b(0,2)	c(0,0) + c(0,1) + c(0,2)										
a(0,0) + a(0,1)	b(0,1)	c(0,0) + c(0,1) + c(0,2)										
a(0,0) + a(0,1) + a(0,2)	b(0,1) + b(0,2)	c(0,2)										
h1	<table><tr><td></td><td>b(1,0) + b(1,1) + b(1,2)</td><td></td></tr></table>		b(1,0) + b(1,1) + b(1,2)		<table><tr><td></td><td></td><td>c(1,0) + c(1,1) + c(1,2)</td></tr></table>			c(1,0) + c(1,1) + c(1,2)	<table><tr><td>a(1,0) + a(1,1) + a(1,2)</td><td></td><td></td></tr></table>	a(1,0) + a(1,1) + a(1,2)		
	b(1,0) + b(1,1) + b(1,2)											
		c(1,0) + c(1,1) + c(1,2)										
a(1,0) + a(1,1) + a(1,2)												
h2	<table><tr><td></td><td>b(2,0) + b(2,1) + b(2,2)</td><td></td></tr></table>		b(2,0) + b(2,1) + b(2,2)		<table><tr><td></td><td></td><td>c(2,0) + c(2,1) + c(2,2)</td></tr></table>			c(2,0) + c(2,1) + c(2,2)	<table><tr><td>a(2,0) + a(2,1) + a(2,2)</td><td></td><td></td></tr></table>	a(2,0) + a(2,1) + a(2,2)		
	b(2,0) + b(2,1) + b(2,2)											
		c(2,0) + c(2,1) + c(2,2)										
a(2,0) + a(2,1) + a(2,2)												

纵向all-reduce:



横向all-gather:



其他优化：

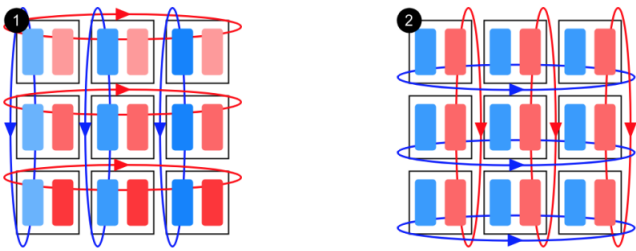
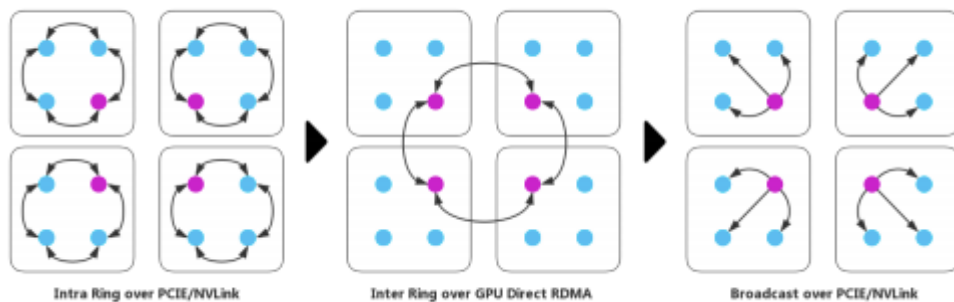


Figure 4: 2-D all-reduce across a hypothetical 3×3 torus. In the first phase (left), the blue tensors are summed along the Y dimension while the red tensors are summed concurrently along the X dimension. In the second phase (right), the dimensions are flipped, which completes the all-reduce for both halves.

利用章节1的计算方法，假设有 p^2 个节点，则通讯cost为 $4(p-1)a + 2(1-1/(p^2))nb$

5. three-phase all-reduce

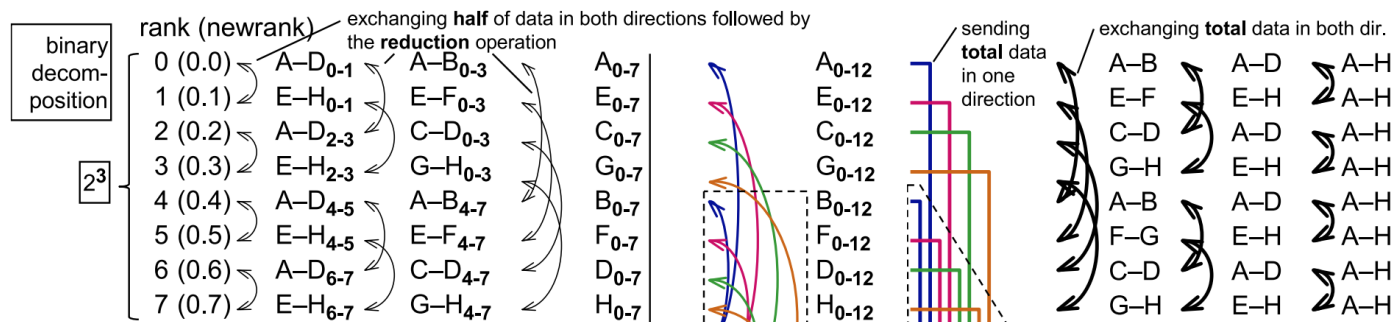


首先在节点内进行all-reduce, 然后在节点间all-reduce, 最后节点内broadcast

假设有k个节点, 每个节点p个卡, 则通讯cost为: $2(k-1)a + 2(p-1)a' + 2(1-1/k)nb + 2(1-1/p)nb' + nb'$

6. recursive doubling

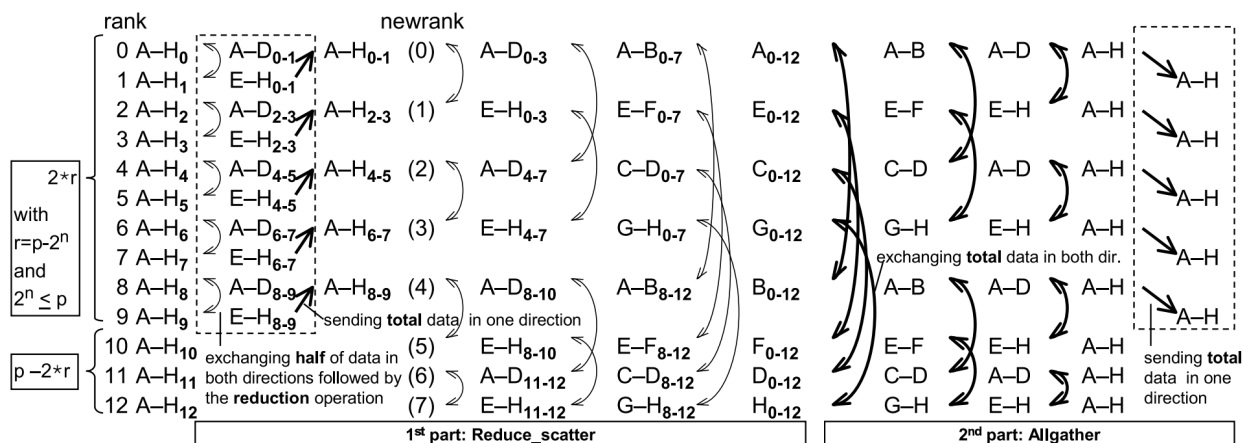
recursive doubling适合类似nv-switch, 且节点数为2的指数幂的场景



上图中0-7包括8个节点, 每个节点的数据分为A-H共8份。

假设有p个卡, 则通讯cost为: $2 * \lg(p) * a + 2 * (1 - 1/p)nb$

对于非2的指数幂, 可以按如下图进行:



先凑出一个2的指数幂，然后按recursive doubling进行计算，其cost为：

$$2 * \lg(p) * a + 2 * (1 - 1/p)nb + 2nb$$

非2的指数幂也可以按照下图进行

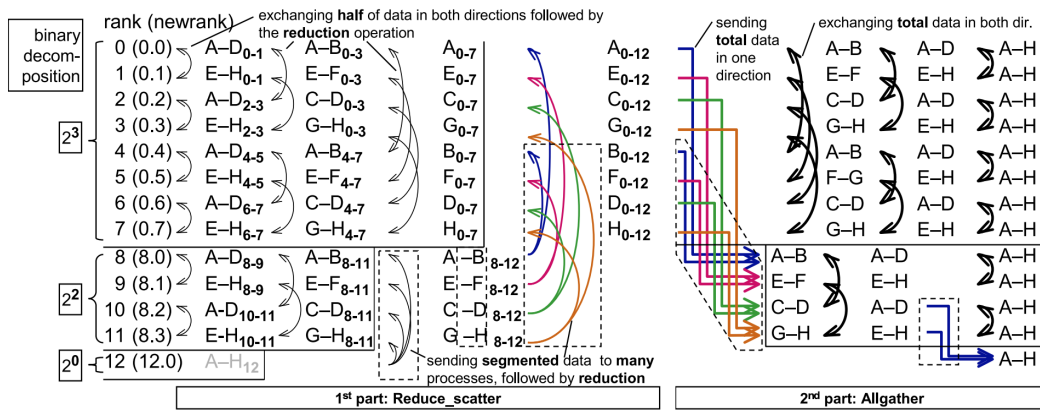
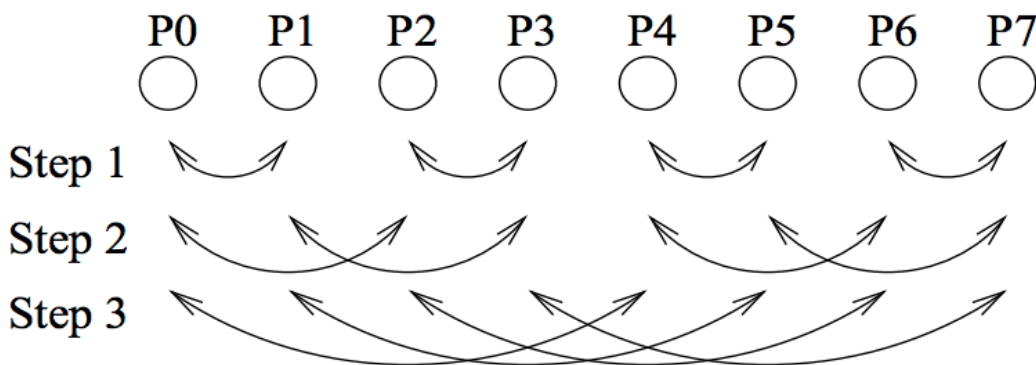


Figure 13: Allreduce using the binary blocks algorithm

cost为： $2\lg(p) a + 2 * (1 - 1/p)nb + nb(1 + 1/2 + \dots + 1/p)$

7. binary tree 算法



相邻两个节点交换，然后0和2， 1和3，以此类推的交换数据。该算法适合任意两个gpu卡之间通讯差别不大，且传输数据量不大的情况

假设p个节点，其代价是： $\lg p (a + nb)$,

8. all-reduce 其他优化

8.1 fusion

all reduce的数据量不大时，可以考虑将多个all-reduce的数据fusion成一个，然后一起all-reduce

8.2 hybrid all-reduce

不同的all-reduce算法适合不同的场景，可以根据具体的规模把不同的all-reduce算法合在一起使用

8.3 pipeline

all-reduce设计到网络和数据拷贝，可以把这些过程和计算op pipeline起来。让GPU同时进行计算和all-reduce